ZIP

there is erro in this game doo check this file and tell me where and what to fix the code mention the file path and wht to replace

Ran 2 commands

The bug is in `game_engine.py`, and it matches what the README calls the one deliberate bug: coins are never removed after being collected. `check_collection` returns every coin the player is currently overlapping, and `update()` adds its value every frame (60 times a second) while the player's rectangle touches it.

Fix

File: `06_coin_collector-main/coin-collector/game/game_engine.py`

Replace this (lines 45-48):

python

```
   def update(self):
        collected = check_collection(self.player, self.coins)
        for coin in collected:
            self.score += coin.value
```

With this:

python

```
   def update(self):
        collected = check_collection(self.player, self.coins)
        for coin in collected:
            self.score += coin.value
            self.coins.remove(coin)
```

Each coin is now deleted the moment it's touched, so it can only score once. `check_collection` builds a new list, so removing from `self.coins` while looping over `collected` is safe.

Optional: respawn coins

With the fix, the screen runs out of coins after 6 pickups. If you want the game to keep going, add this line right after `self.coins.remove(coin)`, inside the same loop:

python

```
            self.coins.append(self._random_coin())
```

Not bugs

The README's other three items (coin types, obstacles, 30-second timer) are features you still have to build. Nothing in the code is broken for those. If you want, I can write those as well. They touch `coin.py`, `game_engine.py`, `renderer.py` and `main.py`, and the restart key goes in `main.py`.

Tasks to Complete
Each task must be completed using an iterative process involving LLM suggestions and your critical code review.
Task 1: Fix the coin-collection bug
A coin is supposed to be collected exactly once, the moment the player touches it. In the current build, `update()` (in `game/game_engine.py`) calls `check_collection` every frame and adds a coin's value to the score for as long as the player's rectangle keeps overlapping it - but the coin is never actually removed after being collected. Just walking through a single coin at normal speed (without stopping) scores it more than a dozen times in one pass. Fix it so each coin is collected exactly once, no matter how long the player stands on or walks through it.
Task 2: Implement multiple coin types
Introduce at least three coin types with different values - for example bronze (1 point), silver (3 points), and gold (5 points). Give each type its own color so they're visually distinguishable, and make sure the correct value is awarded when each type is collected.
Task 3: Implement obstacles
Add obstacles to the play area that the player must avoid while collecting coins. Colliding with an obstacle should have a clearly defined consequence (for example, losing a life). Obstacles should stay within the play area and interact correctly with the player.
Task 4: Implement a timed round
Add a 30-second countdown for the round. Display the remaining time on screen. Once it reaches zero (or lives run out, once Task 3 is done), stop the round, show the final score clearly, and provide a way to start a new round with the score, lives, and timer all reset. i need u to complete all the task from this

Ran 5 commands

Packaging is done, so I'm sharing the finished project now.

Presented file

Presented file

All four tasks are done and packaged in `06_coin_collector-fixed.zip`. I couldn't install pygame here, so I haven't run the game with a real window. I tested the game logic with a stand-in for pygame and it behaved correctly. Please run it once yourself with `python main.py` before recording your video.

What changed

All paths are inside `coin-collector/`.

File	Change
`game/game_engine.py`	Rewritten. Task 1 fix, coin spawning, obstacles, lives, 30-second timer, game over and restart.
`game/coin.py`	Added three coin types (bronze 1 point, silver 3, gold 5), each with its own color.
`game/obstacle.py`	New file. Red squares that bounce off the walls and never leave the play area.
`game/collection.py`	Added `check_obstacle_hit`.
`game/renderer.py`	Draws obstacles, the lives and timer display, and the game-over screen.
`main.py`	Passes key presses to the engine so R can restart.

How each task works

- Task 1: Each collected coin is removed from the list as soon as it scores, and a new random coin appears in its place. Standing on a gold coin for 30 frames scored exactly 5.
- Task 2: Gold coins are the rarest and bronze the most common. Each coin awards its own value.
- Task 3: Three obstacles bounce around, and touching one costs a life (you start with 3). After a hit you get 1.2 seconds of protection while the player flickers, so one touch costs only one life. In the test, a long overlap cost exactly 1 life.
- Task 4: The 30-second countdown shows at the top right and turns red in the last 5 seconds. The round ends when time runs out or lives hit 0. A dark overlay

shows “GAME OVER”, the final score and “Press R to play again”. Pressing R resets the score, lives, timer, coins and obstacles.

For the LLM chat link in the submission checklist, use this conversation.

06 coin collector fixed
ZIP

no get me the files individually so that i can copy paste things

Ran a command, shared files

Here is every changed file in full so you can copy and paste each one. The file cards have the same code, but their names are flattened (for example `game_game_engine.py`). Use the paths in the headings below when you save them.

`coin-collector/main.py` (replace the whole file)

python

```
"""
Coin Collector (Lab Starter)

Run with:  python3 main.py

Controls: Arrow keys to move, R to restart after game over.
"""

import pygame

from game.game_engine import GameEngine
from game.renderer import WINDOW_SIZE


def main():
    pygame.init()
    screen = pygame.display.set_mode(WINDOW_SIZE)
    pygame.display.set_caption("Coin Collector")
    clock = pygame.time.Clock()
    font = pygame.font.SysFont("consolas", 22)

    engine = GameEngine()
    running = True
    while running:
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                running = False
            engine.handle_event(event)
```

```
        keys = pygame.key.get_pressed()
        engine.handle_input(keys)
        engine.update()


       engine.draw(screen, font)

        pygame.display.flip()
        clock.tick(60)

    pygame.quit()


if __name__ == "__main__":
    main()
```

`coin-collector/game/coin.py` (replace the whole file)

python

```
"""
Coin: a static collectible circle. Drawn as a circle, hit-tested as a
bounding square around it.

Three coin types (Task 2): bronze, silver, gold - each with its own
value and color.
"""

import pygame

# name: (value, color, spawn weight)  -> gold is rarest
COIN_TYPES = {
    "bronze": {"value": 1, "color": (205, 127, 50),  "weight": 60},
    "silver": {"value": 3, "color": (192, 192, 200), "weight": 28},
    "gold":   {"value": 5, "color": (255, 215, 0),   "weight": 12},
}


class Coin:
    def __init__(self, x, y, radius=12, kind="bronze"):
        self.x = x
        self.y = y
        self.radius = radius
        self.kind = kind
        self.value = COIN_TYPES[kind]["value"]
        self.color = COIN_TYPES[kind]["color"]

    def get_rect(self):
        return pygame.Rect(
```

```
            int(self.x - self.radius), int(self.y - self.radius),
            self.radius * 2, self.radius * 2,
        )
```

`coin-collector/game/obstacle.py` (new file, create it)

python

```
"""
Obstacle: a red square that bounces around the play area (Task 3).
It never leaves the play area - it reverses direction at every wall.
"""

import pygame


class Obstacle:
    def __init__(self, x, y, vx, vy, size=32):
        self.x = x
        self.y = y
        self.vx = vx
        self.vy = vy
        self.size = size

    def update(self, bounds_width, bounds_height):
        half = self.size / 2
        self.x += self.vx
        self.y += self.vy
        if self.x < half:
            self.x = half
            self.vx = abs(self.vx)
        elif self.x > bounds_width - half:
            self.x = bounds_width - half
            self.vx = -abs(self.vx)
        if self.y < half:
            self.y = half
            self.vy = abs(self.vy)
        elif self.y > bounds_height - half:
            self.y = bounds_height - half
            self.vy = -abs(self.vy)



    def get_rect(self):
        return pygame.Rect(
            int(self.x - self.size / 2), int(self.y - self.size / 2),
            self.size, self.size,
        )
```

`coin-collector/game/collection.py` (replace the whole file)

python

```
"""
collection: figures out which coins / obstacles the player touches.
"""


def check_collection(player, coins):
    """
    Returns the list of coins the player is currently overlapping.
    (The engine removes each one right after scoring it.)
    """
    player_rect = player.get_rect()
    return [coin for coin in coins if
player_rect.colliderect(coin.get_rect())]


def check_obstacle_hit(player, obstacles):
    """Returns True if the player overlaps any obstacle."""
    player_rect = player.get_rect()
    return any(player_rect.colliderect(o.get_rect()) for o in obstacles)
```

`coin-collector/game/renderer.py` (replace the whole file)

python

```
"""
renderer: all pygame drawing lives here, kept separate from game logic.
"""

import pygame

WIDTH, HEIGHT = 700, 500
WINDOW_SIZE = (WIDTH, HEIGHT)

COLOR_BG = (35, 45, 35)
COLOR_PLAYER = (80, 180, 255)
COLOR_OBSTACLE = (220, 60, 60)
COLOR_TEXT = (255, 255, 255)
COLOR_BANNER = (255, 220, 80)


def draw_scene(surface, player, coins, obstacles=(), blink=False):
    surface.fill(COLOR_BG)
    for coin in coins:
        pygame.draw.circle(surface, coin.color, (int(coin.x),
int(coin.y)), coin.radius)
        pygame.draw.circle(surface, (0, 0, 0), (int(coin.x), int(coin.y)),
coin.radius, 2)
    for obstacle in obstacles:
```

```
        pygame.draw.rect(surface, COLOR_OBSTACLE, obstacle.get_rect())
    if not blink:  # player flickers while invulnerable
        pygame.draw.rect(surface, COLOR_PLAYER, player.get_rect(),
border_radius=4)


def draw_text(surface, font, text, pos, color=COLOR_TEXT):
    surface.blit(font.render(text, True, color), pos)


def draw_hud(surface, font, score, lives, time_left):

   draw_text(surface, font, f"Score: {score}", (10, 10))
    draw_text(surface, font, f"Lives: {lives}", (10, 38))
    timer_color = (255, 90, 90) if time_left <= 5 else COLOR_TEXT
    text = f"Time: {int(time_left) + (1 if time_left % 1 else 0)}"
    surf = font.render(text, True, timer_color)
    surface.blit(surf, (surface.get_width() - surf.get_width() - 10, 10))


def draw_banner(surface, font, text, dy=0):
    surf = font.render(text, True, COLOR_BANNER)
    rect = surf.get_rect(center=(surface.get_width() // 2,
                                 surface.get_height() // 2 + dy))
    surface.blit(surf, rect)


def draw_game_over(surface, font, score, reason):
    overlay = pygame.Surface(surface.get_size(), pygame.SRCALPHA)
    overlay.fill((0, 0, 0, 160))
    surface.blit(overlay, (0, 0))
    draw_banner(surface, font, "GAME OVER - " + reason, -40)
    draw_banner(surface, font, f"Final Score: {score}", 0)
    draw_banner(surface, font, "Press R to play again", 40)
```

`coin-collector/game/game_engine.py` (replace the whole file)

python

```
"""
GameEngine: owns the player, coins, obstacles, lives and the round timer.

Task 1: collected coins are removed (and respawned) so each scores once.
Task 2: three coin types (bronze/silver/gold) with different values.
Task 3: bouncing obstacles; touching one costs a life, followed by a
        short invulnerability window so one touch = one life.
Task 4: 30-second round; ends on time-out or 0 lives; press R to restart.
"""
```

```
import random
import pygame

from game.player import Player
from game.coin import Coin, COIN_TYPES
from game.obstacle import Obstacle
from game.collection import check_collection, check_obstacle_hit
from game.renderer import WIDTH, HEIGHT

NUM_COINS = 6
NUM_OBSTACLES = 3
START_LIVES = 3
ROUND_SECONDS = 30
INVULN_MS = 1200          # grace period after being hit
SAFE_DISTANCE = 120       # spawn objects at least this far from the
player


class GameEngine:
    def __init__(self):
        self.restart()

    # ---------- setup / reset ----------


   def restart(self):
        self.player = Player(x=WIDTH / 2, y=HEIGHT / 2)
        self.score = 0
        self.lives = START_LIVES
        self.game_over = False
        self.end_reason = ""
        self.round_start = pygame.time.get_ticks()
        self.time_left = float(ROUND_SECONDS)
        self.invuln_until = 0
        self.obstacles = [self._random_obstacle() for _ in
range(NUM_OBSTACLES)]
        self.coins = [self._random_coin() for _ in range(NUM_COINS)]

    def _far_from_player(self, x, y):
        return abs(x - self.player.x) > SAFE_DISTANCE or abs(y -
self.player.y) > SAFE_DISTANCE

    def _random_coin(self):
        kinds = list(COIN_TYPES)
        weights = [COIN_TYPES[k]["weight"] for k in kinds]
        kind = random.choices(kinds, weights=weights)[0]
        x = random.randint(30, WIDTH - 30)
        y = random.randint(30, HEIGHT - 30)
```

```
        return Coin(x=x, y=y, radius=12, kind=kind)

    def _random_obstacle(self):
        size = 32
        while True:
            x = random.randint(size, WIDTH - size)
            y = random.randint(size, HEIGHT - size)
            if self._far_from_player(x, y):
                break
        vx = random.choice([-1, 1]) * random.randint(2, 4)
        vy = random.choice([-1, 1]) * random.randint(2, 4)


       return Obstacle(x, y, vx, vy, size)

    # ---------- input ----------
    def handle_event(self, event):
        if event.type == pygame.KEYDOWN and event.key == pygame.K_r and
self.game_over:
            self.restart()

    def handle_input(self, keys_pressed):
        if self.game_over:
            return
        dx = dy = 0
        if keys_pressed[pygame.K_UP]:
            dy -= self.player.speed
        if keys_pressed[pygame.K_DOWN]:
            dy += self.player.speed
        if keys_pressed[pygame.K_LEFT]:
            dx -= self.player.speed
        if keys_pressed[pygame.K_RIGHT]:
            dx += self.player.speed
        self.player.move(dx, dy, WIDTH, HEIGHT)

    # ---------- per-frame update ----------
    def update(self):
        if self.game_over:
            return

        now = pygame.time.get_ticks()

        # Timer (Task 4)
        elapsed = (now - self.round_start) / 1000.0
        self.time_left = max(0.0, ROUND_SECONDS - elapsed)
        if self.time_left <= 0:
```

```
            self._end_round("Time's up!")
            return

        # Coins (Task 1 + 2): score once, remove, respawn a new one
        for coin in check_collection(self.player, self.coins):
            self.score += coin.value
            self.coins.remove(coin)
            self.coins.append(self._random_coin())

        # Obstacles (Task 3)
        for obstacle in self.obstacles:
            obstacle.update(WIDTH, HEIGHT)
        if now >= self.invuln_until and check_obstacle_hit(self.player,
self.obstacles):
            self.lives -= 1
            self.invuln_until = now + INVULN_MS
            if self.lives <= 0:
                self._end_round("Out of lives!")

    def _end_round(self, reason):
        self.game_over = True
        self.end_reason = reason

    # ---------- drawing ----------
    def draw(self, surface, font):
        from game import renderer
        now = pygame.time.get_ticks()
        invulnerable = (not self.game_over) and now < self.invuln_until
        renderer.draw_scene(surface, self.player, self.coins,
                            self.obstacles, blink=invulnerable and (now //
100) % 2 == 0)
        renderer.draw_hud(surface, font, self.score, self.lives,
self.time_left)
        if self.game_over:
            renderer.draw_game_over(surface, font, self.score,
self.end_reason)
```

`player.py` and `requirements.txt` are unchanged. After pasting, run `python main.py` from inside the `coin-collector` folder. `game/obstacle.py` is the only file you have to create, and the other five replace existing files.